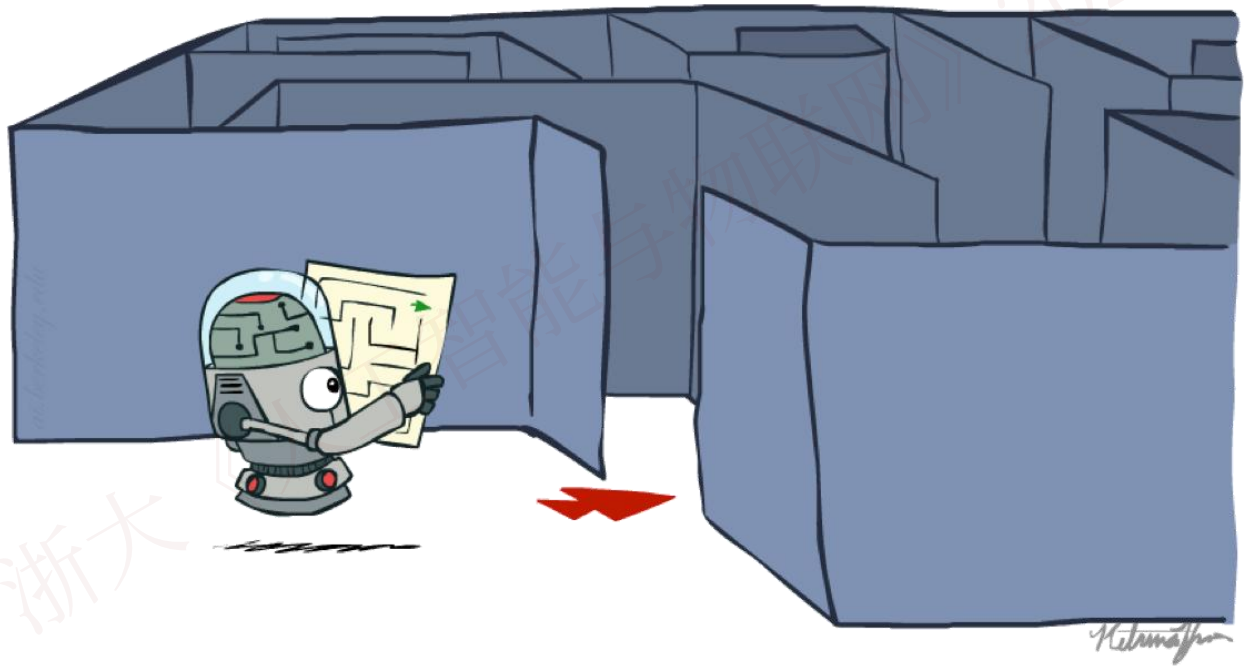
Artificial Intelligence & IoT

Search



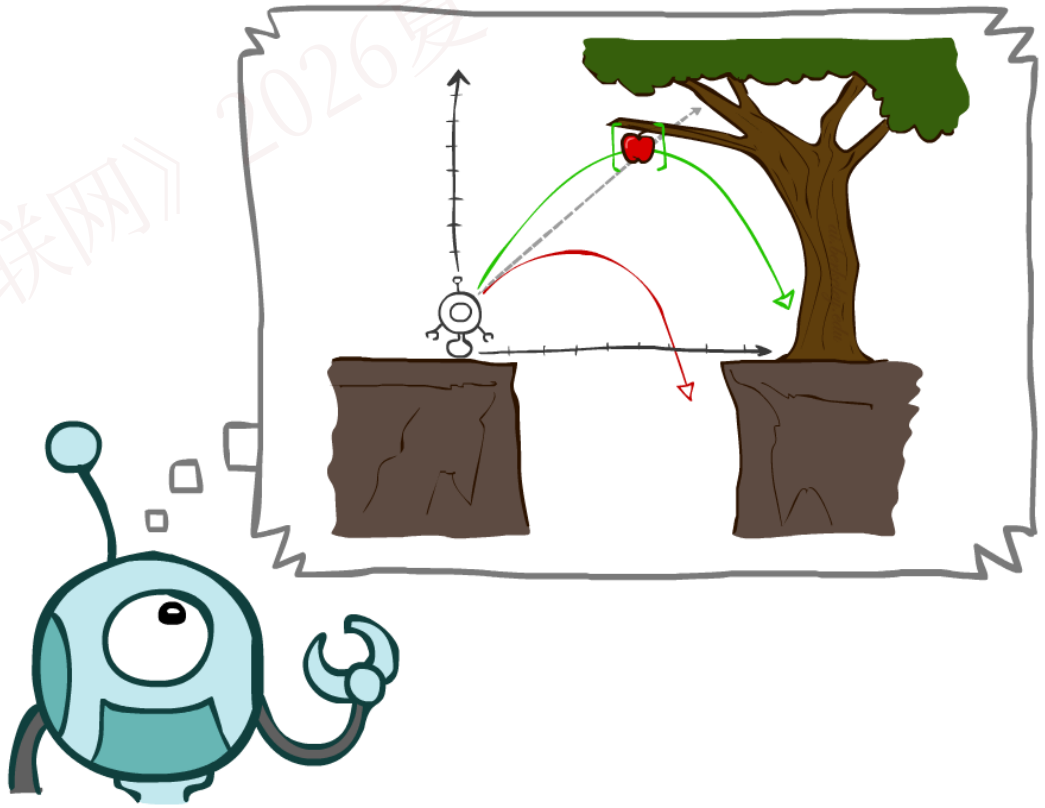
Instructor: Kaikai Pan

Zhejiang University

[These slides were adapted from the slides by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley. http://ai.berkeley.edu.]

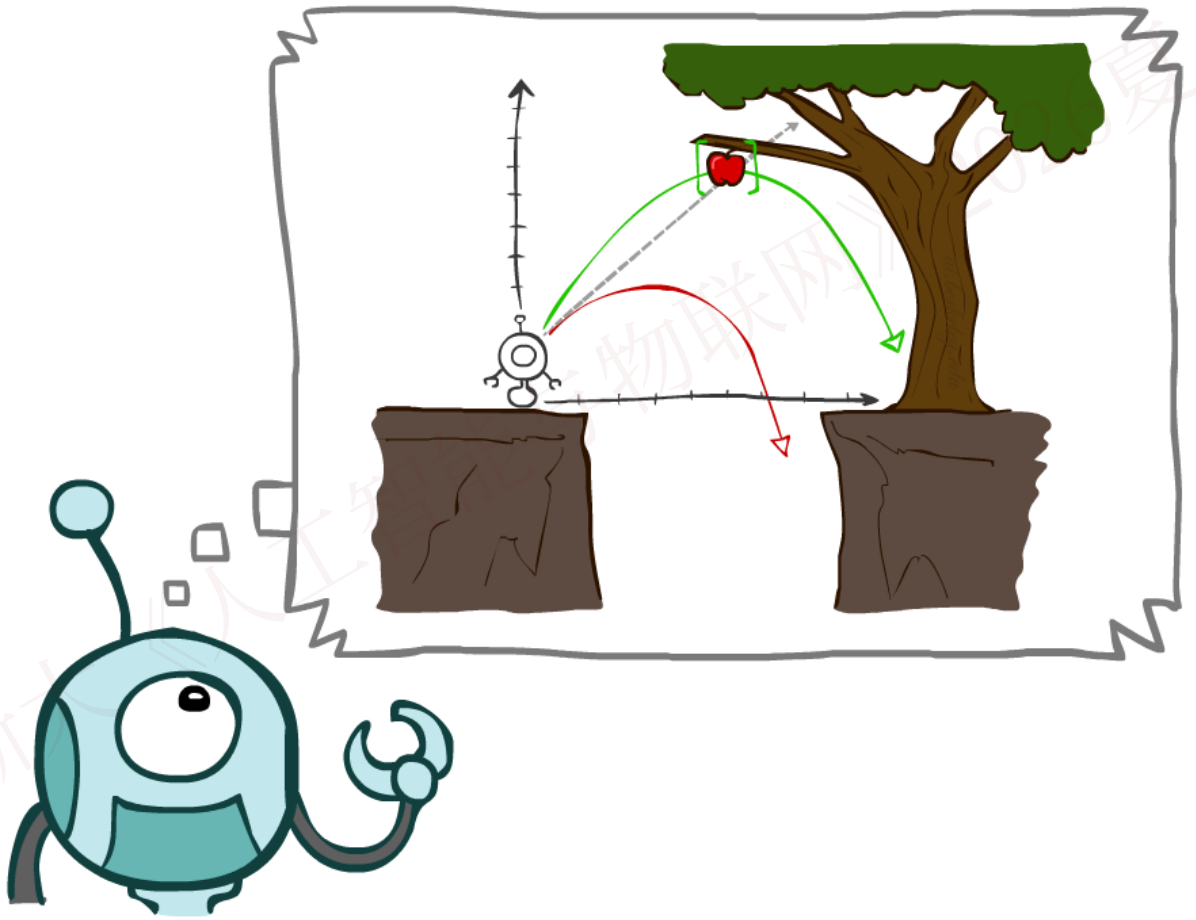
Today

- Agents that Plan Ahead
- Search Problems
- Uninformed Search Methods
  - Depth-First Search 深度优先搜索
  - Breadth-First Search 宽度优先搜索
  - Uniform-Cost Search 一致代价搜索



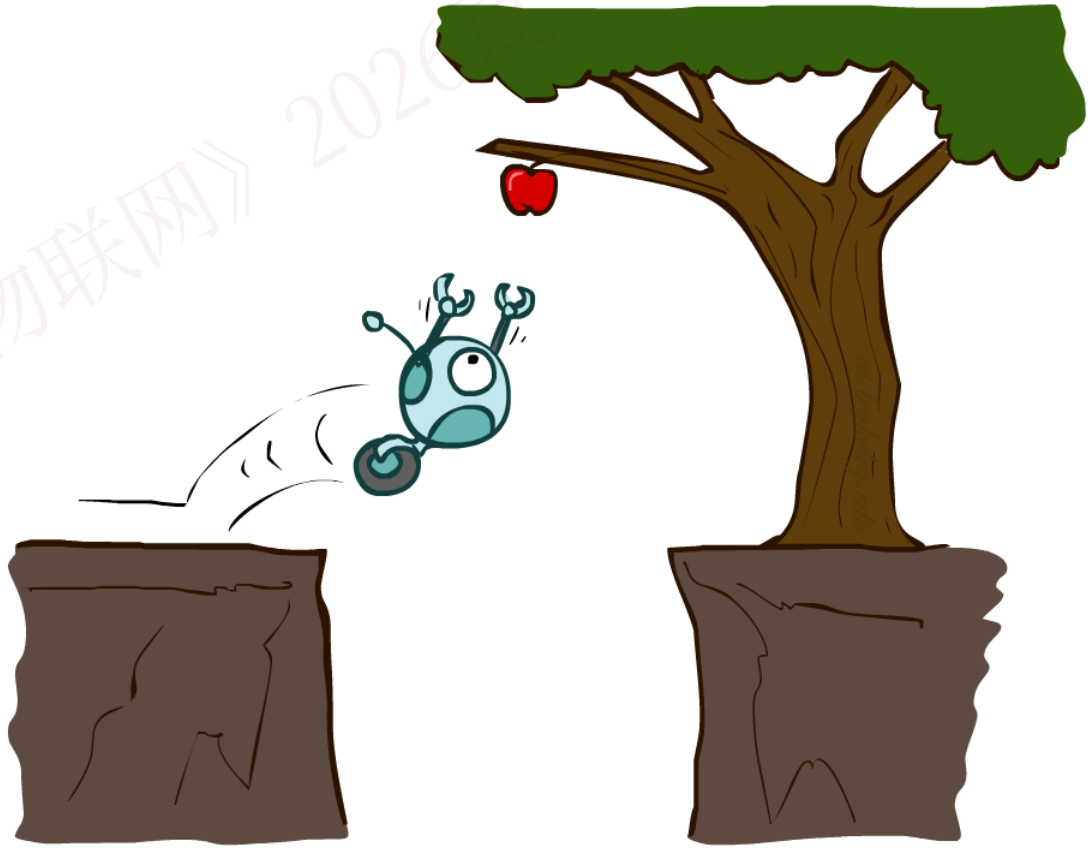
浙大《人工智能与物联网》2026夏

Agents that Plan



Reflex Agents

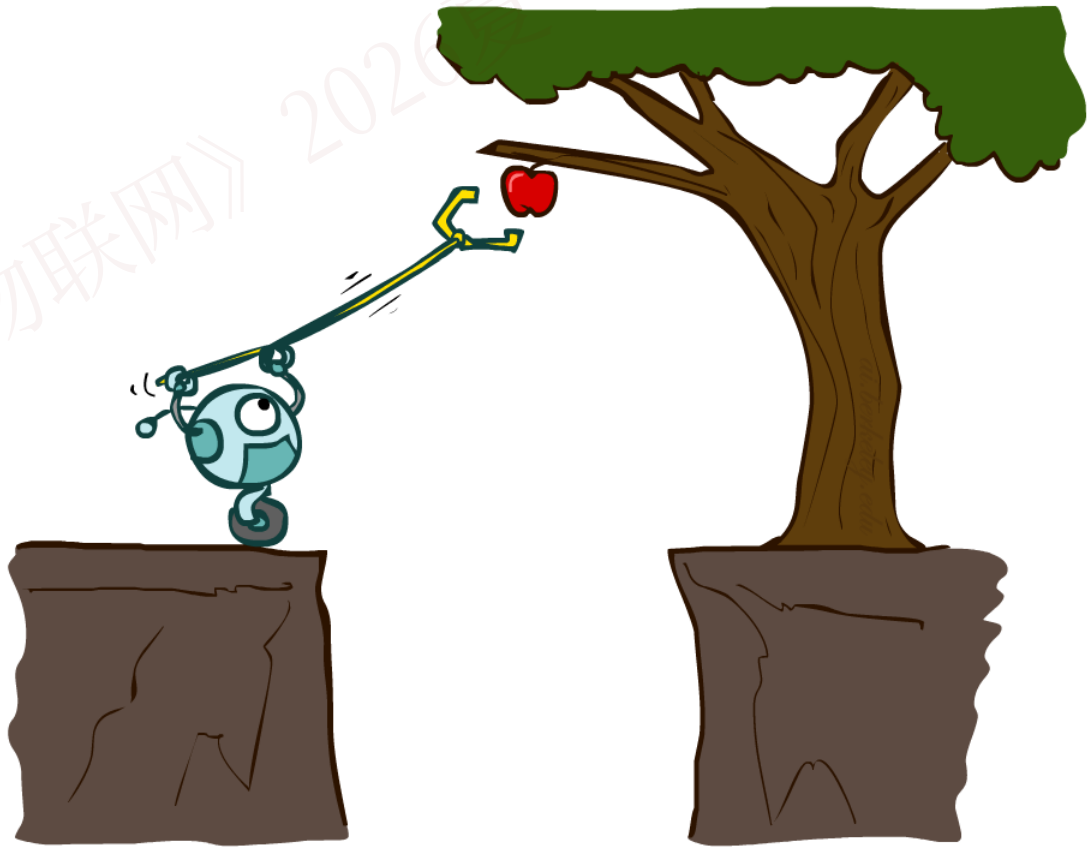
- Reflex (反射) agents:
  - Choose the action based on the current percept (and maybe memory)
  - May have memory or a model of the world's current state
  - Do not consider the future consequences of their actions
  - Consider how the world IS
- Can a reflex agent be rational?



浙大《人工智能与物联网》2026春

Planning Agents

- Planning (规划) agents:
  - Ask “what if”
  - Decisions based on (hypothesized) consequences of actions
  - Must have a model of how the world evolves in response to actions
  - Must formulate a goal (test)
  - Consider how the world WOULD BE
- Optimal (最优) vs. complete (完备) planning
- Planning vs. replanning



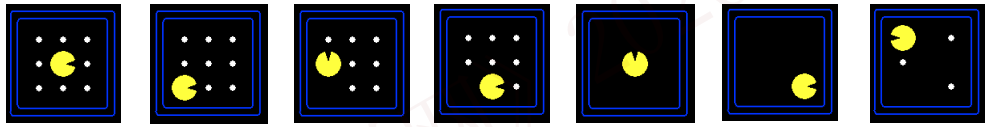
浙大《人工智能与物联网》2026春

Search Problems

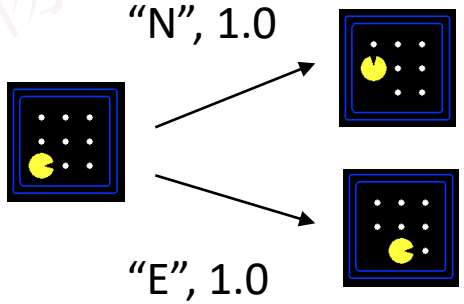


Search Problems

- A search problem (搜索问题) consists of:
  - A state space
    (状态空间)



  - A successor function
    (with actions, costs)
    (转移函数, 行动, 代价)

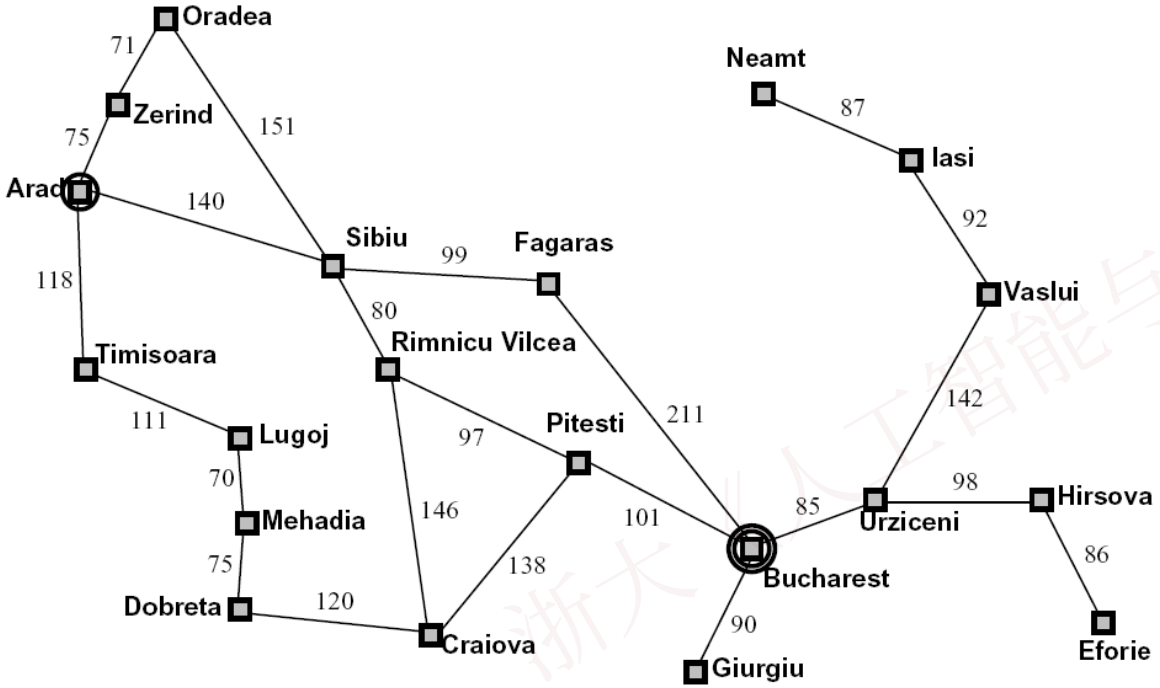


  - A start state and a goal test
    (初始状态, 目标测试)
- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

Search Problems Are Models



Example: Traveling Problem

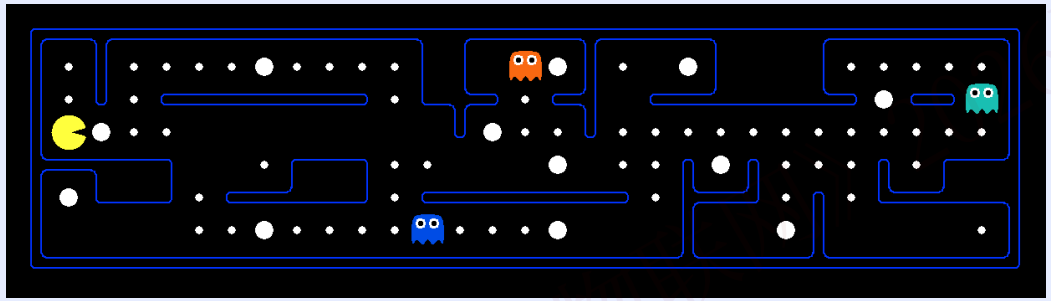


- State space:
  - Cities
- Successor function:
  - Roads: Go to adjacent city with cost = distance
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?
- Solution?

浙大人工智能与物联网公 2026夏

What's in a State Space?

The world state includes every last detail of the environment



A search state keeps only the details needed for planning (abstraction)

- Problem: Pathing
  - States: (x,y) location
  - Actions: ↑↓←→
  - Successor: update location only
  - Goal test: is (x,y)=END

- Problem: Eat-All-Dots
  - States: {(x,y), dot booleans}
  - Actions: ↑↓←→
  - Successor: update location and possibly a dot boolean
  - Goal test: dots all false

State Space Sizes?

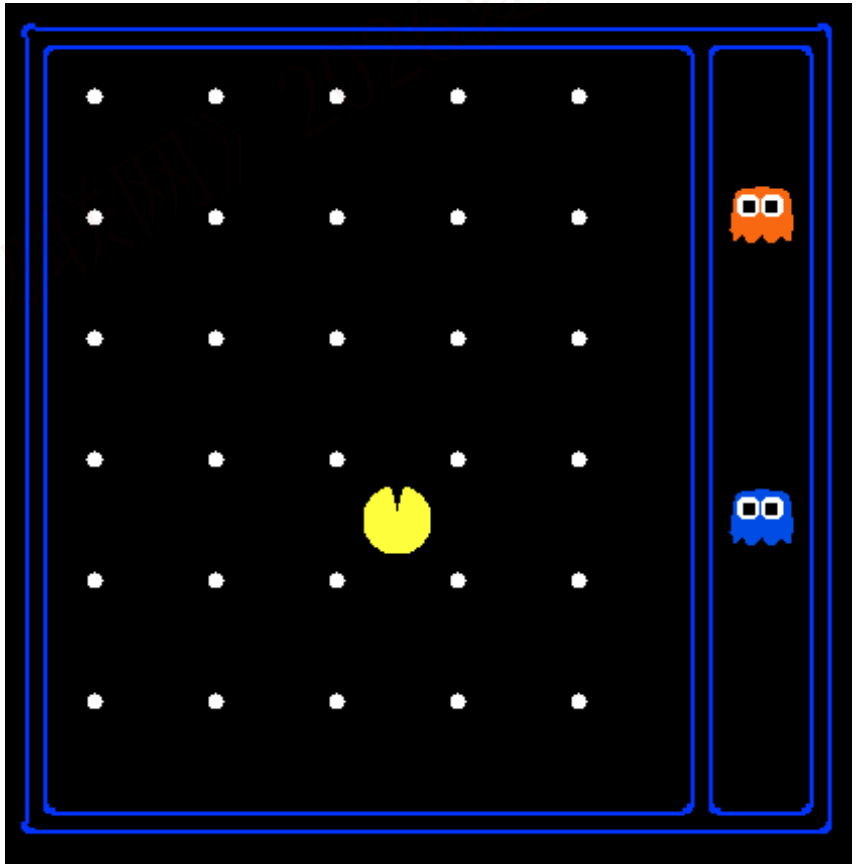
- World state:
  - Agent positions: 120
  - Food count: 30
  - Ghost positions: 12
  - Agent facing: ↑↓←→

- How many
  - World states?

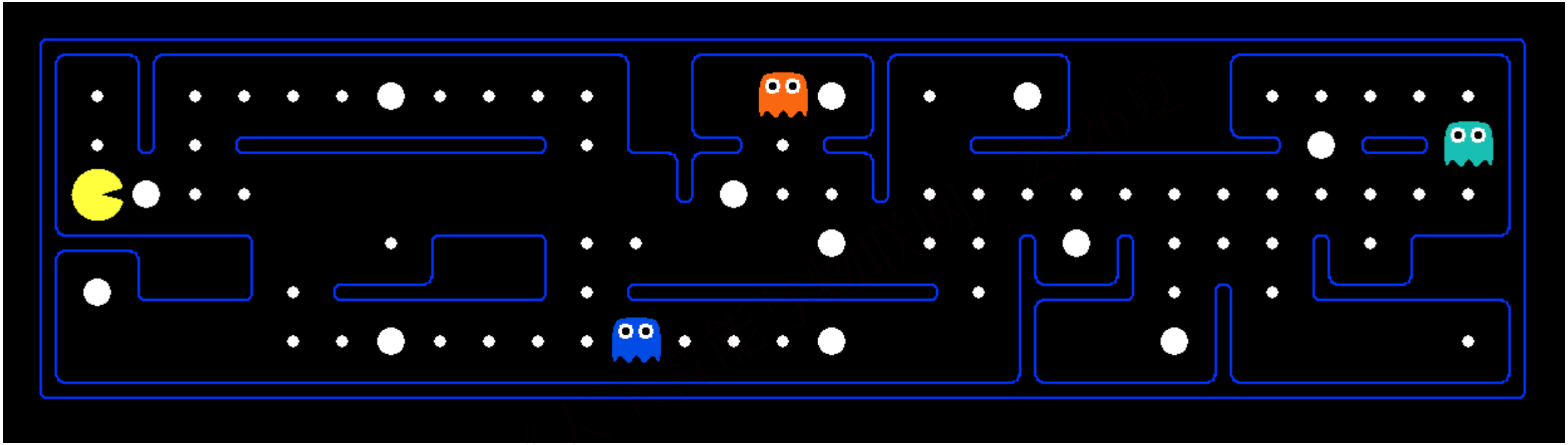
    $120x(2^{30})x(12^2)x4$
  - States for pathing?

    120
  - States for eat-all-dots?

    $120x(2^{30})$

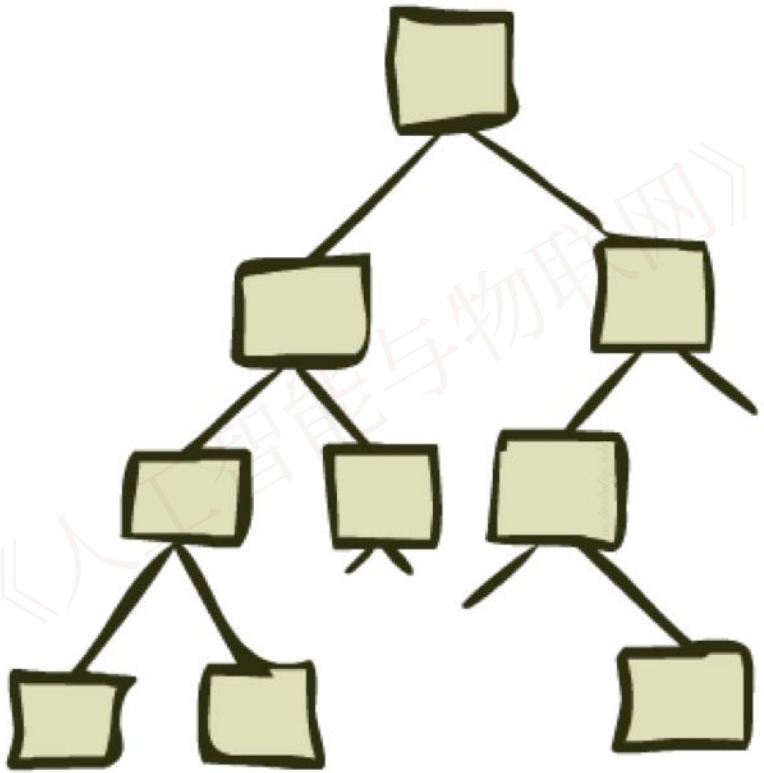


Quiz: Safe Passage



- Problem: eat all dots while keeping the ghosts permanently scared?
  - The ghost will be scared (cannot eat the agent) for a certain period of time after the agent eats a power dot
- What does the state space have to specify?
  - (agent position, dot booleans, power dot booleans, remaining scared time)

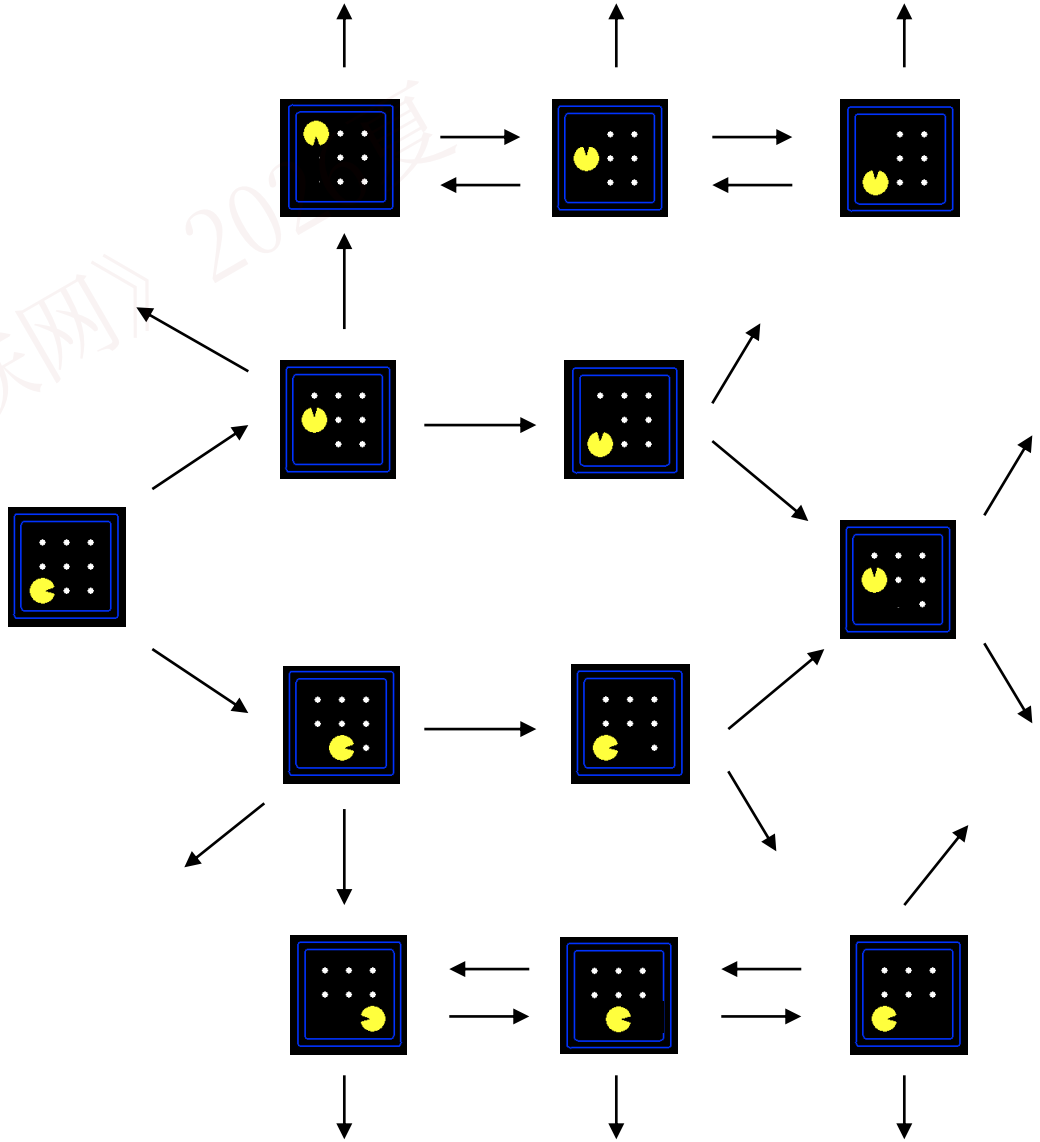
State Space Graphs and Search Trees



浙大《人工智能与物联网》2026夏

State Space Graphs

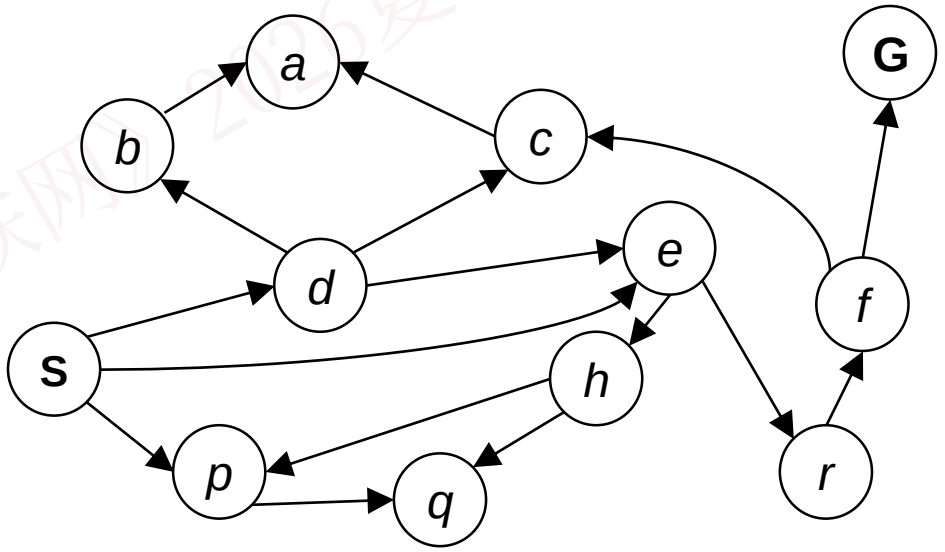
- State space graph (状态空间图): A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Edges represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



浙大《人工智能与物联网》2021

State Space Graphs

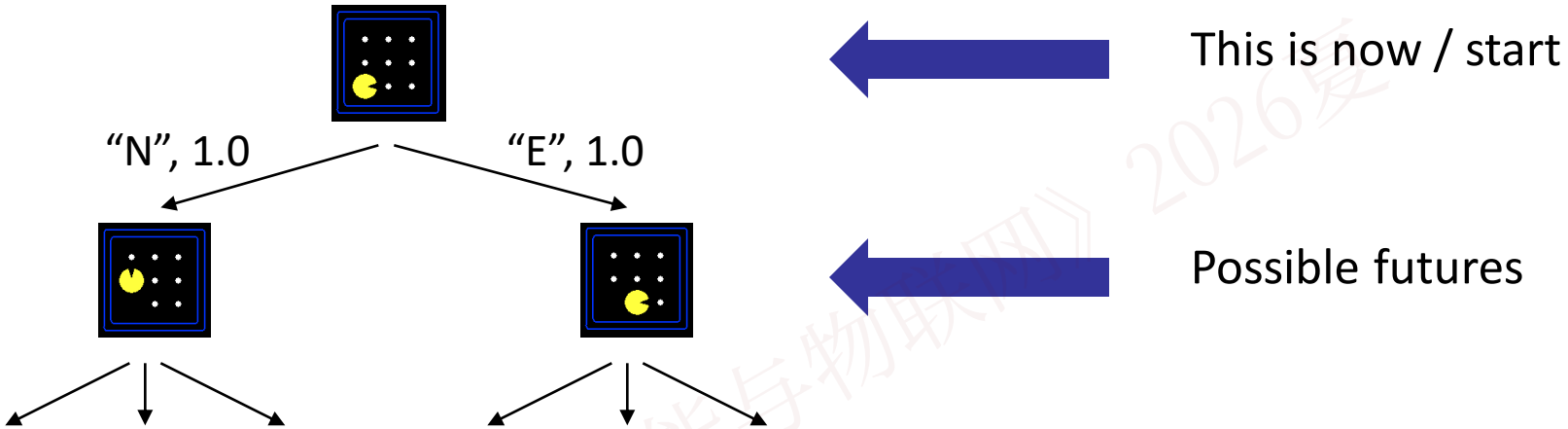
- State space graph (状态空间图): A mathematical representation of a search problem
  - Nodes are (abstracted) world configurations
  - Edges represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
- In a state space graph, each state occurs only once!
- We can rarely build this full graph in memory (it's too big), but it's a useful idea



Tiny state space graph for a "tiny" search problem

浙大《人工智能与物联网》2026夏

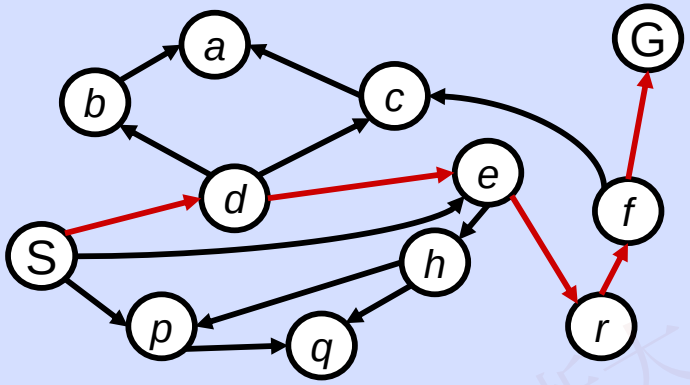
Search Trees



- A search tree (搜索树):
  - A "what if" tree of plans and their outcomes
  - The start state is the root node (根节点)
  - Children (子节点) correspond to successors
  - Nodes show states, but correspond to PLANS that achieve those states
  - For most problems, we can never actually build the whole tree

State Space Graphs vs. Search Trees

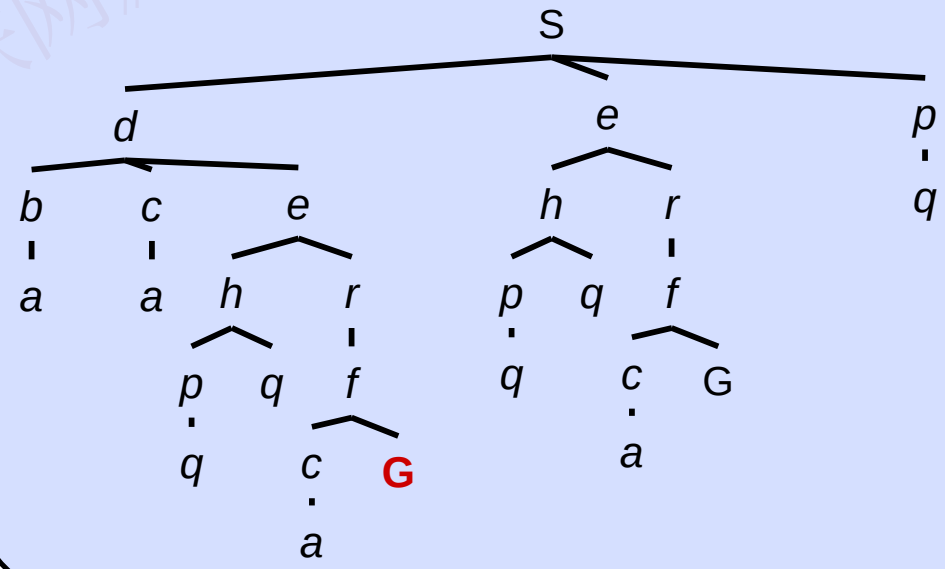
State Space Graph



Each NODE in in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.

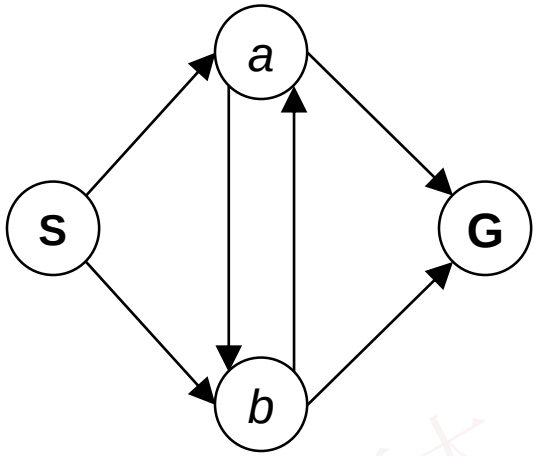
Search Tree



浙大人工智能与物联网》2026夏

Quiz: State Space Graphs vs. Search Trees

Consider this 4-state graph:



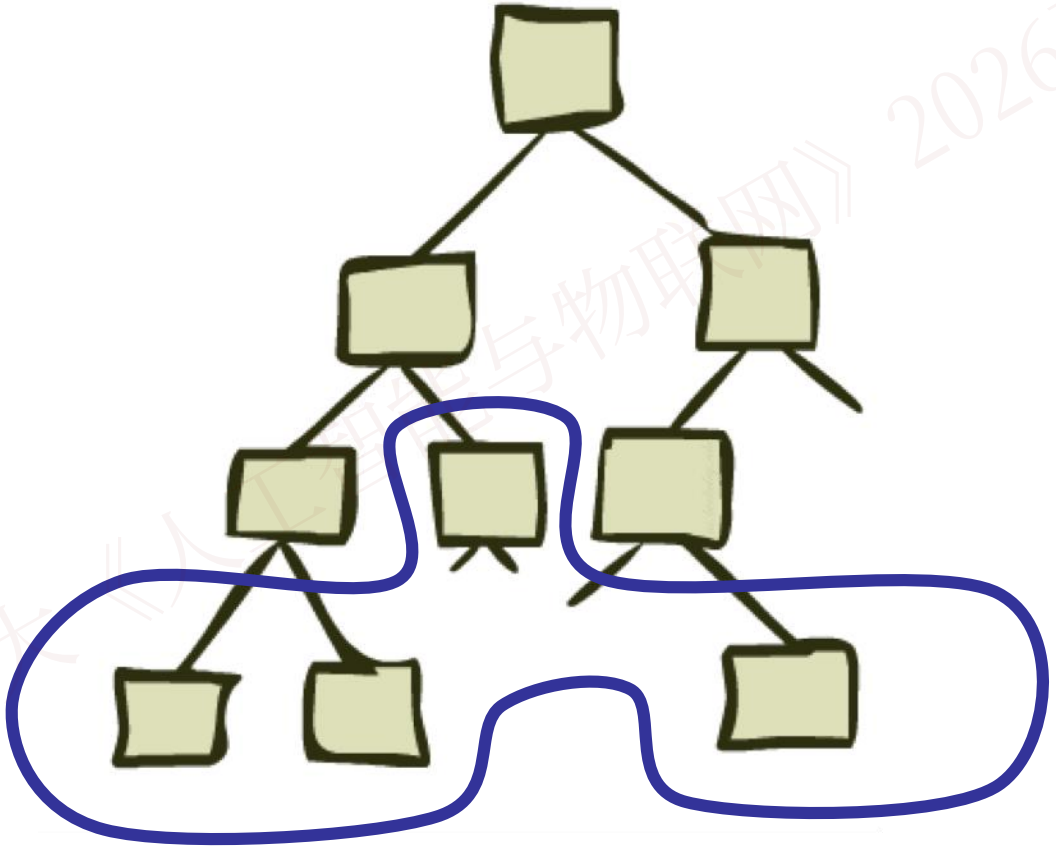
How big is its search tree (from S)?



Important: Lots of repeated structure in the search tree!

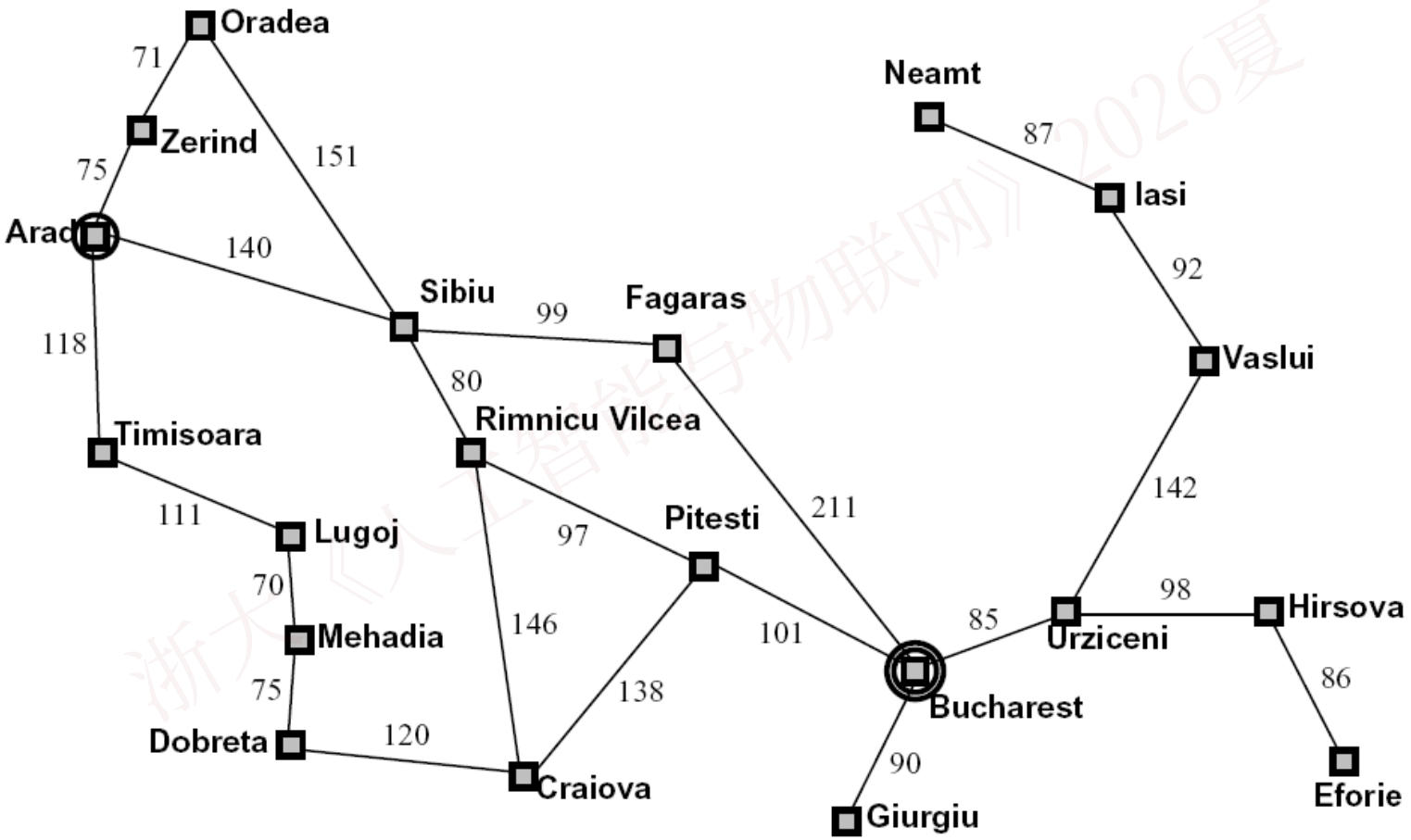
浙大《人工智能与物联网》2020夏

Tree Search

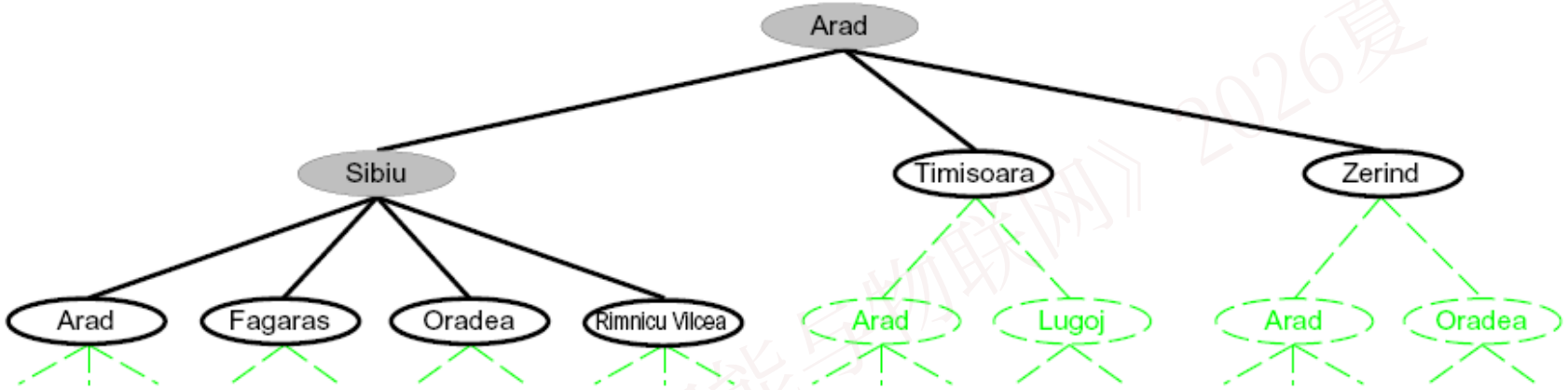


浙大《人工智能与物联网》2026夏

Search Example: Travelling Problem



Searching with a Search Tree



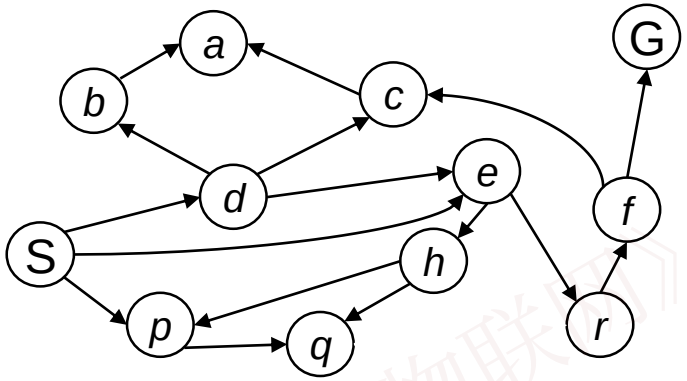
- Search:
  - Expand (扩展) out potential plans (tree nodes)
  - Maintain a fringe (边缘) of partial plans under consideration
  - Try to expand as few tree nodes as possible

General Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

- Important ideas:
  - Fringe (**边缘**)
  - Expansion (**扩展**)
  - Search strategy (**搜索策略**)
- Main question: which fringe nodes to explore?

Example: Tree Search



浙大《人工智能与物联网》2026夏

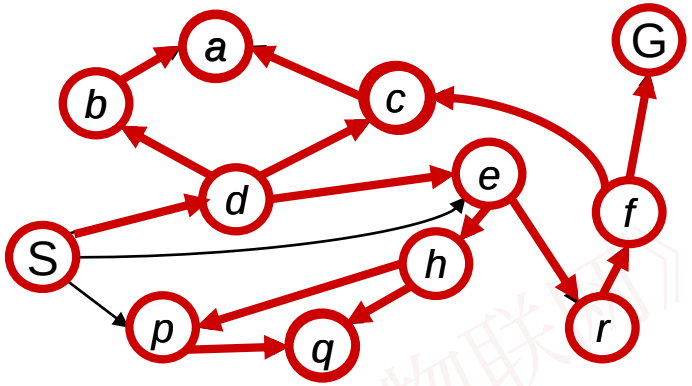
Depth-First Search

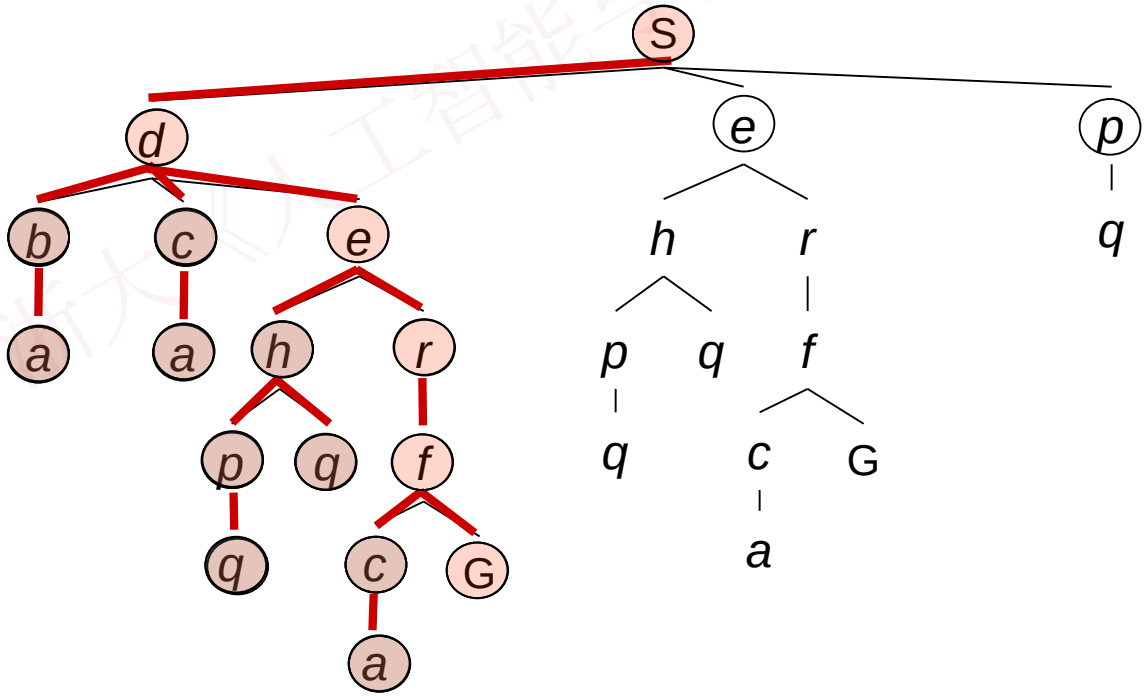


Depth-First Search

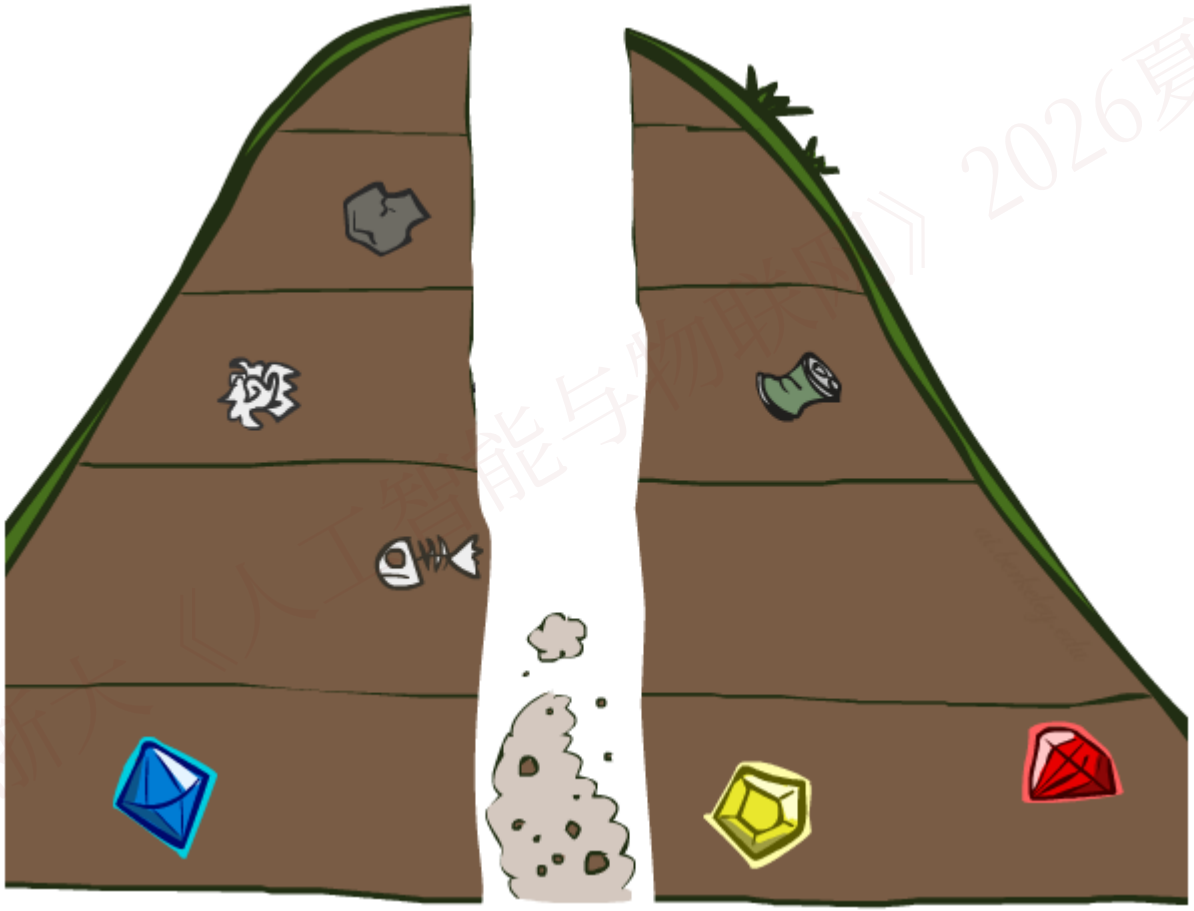
Strategy: expand a deepest node first

Implementation: Fringe is a LIFO stack





Search Algorithm Properties



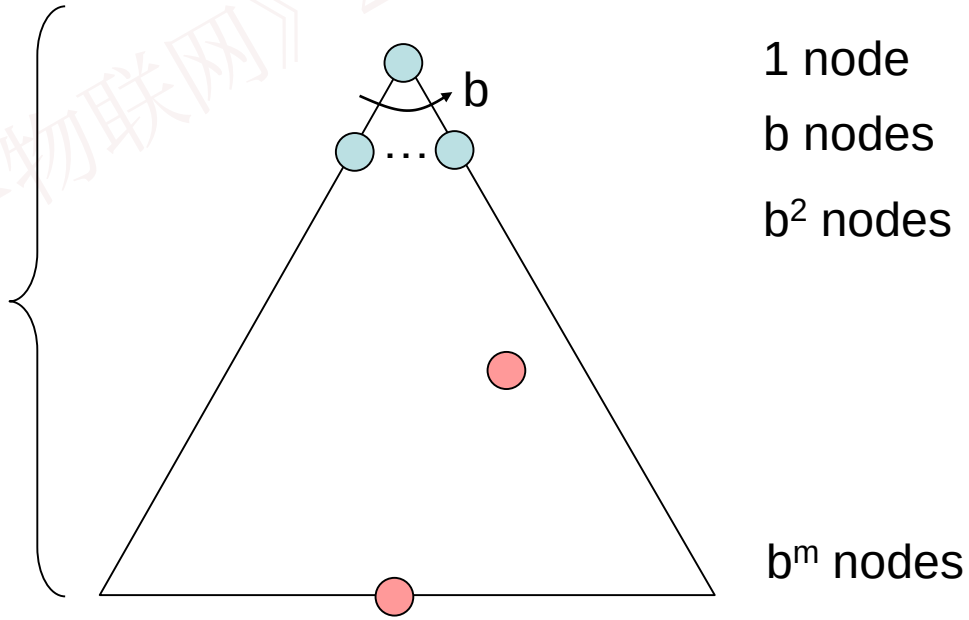
浙大人工智能与物联网 2026夏

Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists? (finite, yes)
- Optimal: Guaranteed to find the least cost path? (no)
- Time complexity?
- Space complexity?

- A search tree:
  - b is the branching factor (分支因子)
  - m is the maximum depth (最大深度)
  - solutions at various depths

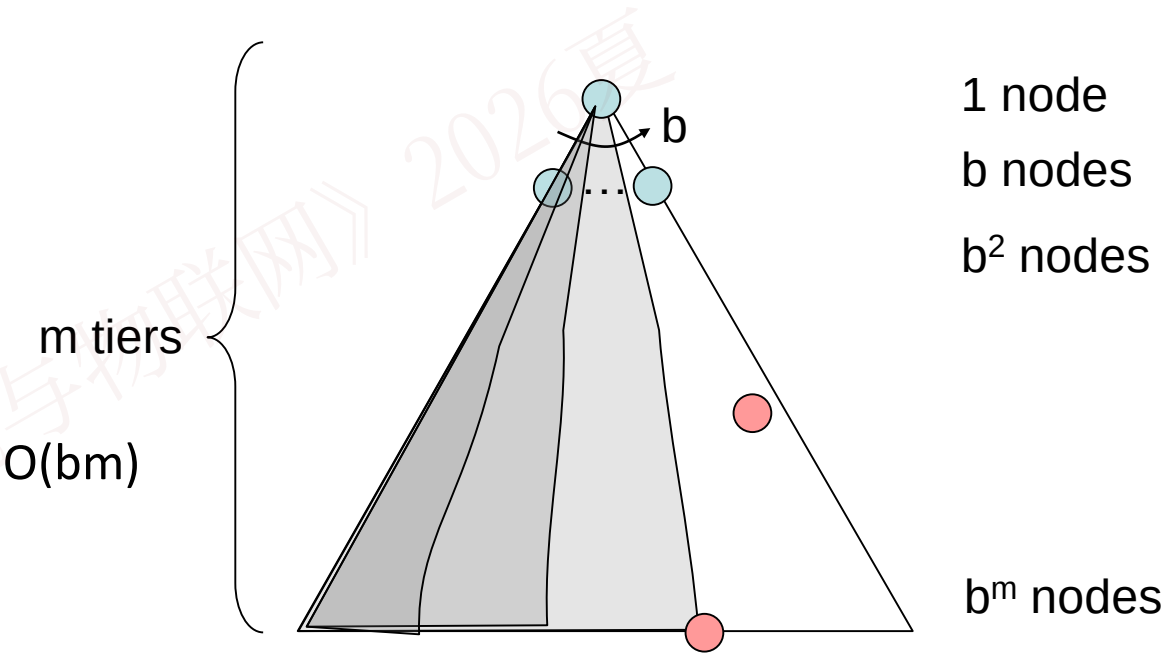
- Number of nodes in entire tree?
  - $1 + b + b^2 + \ldots. b^m = O(b^m)$



浙大《人工智能与物联网》2026夏

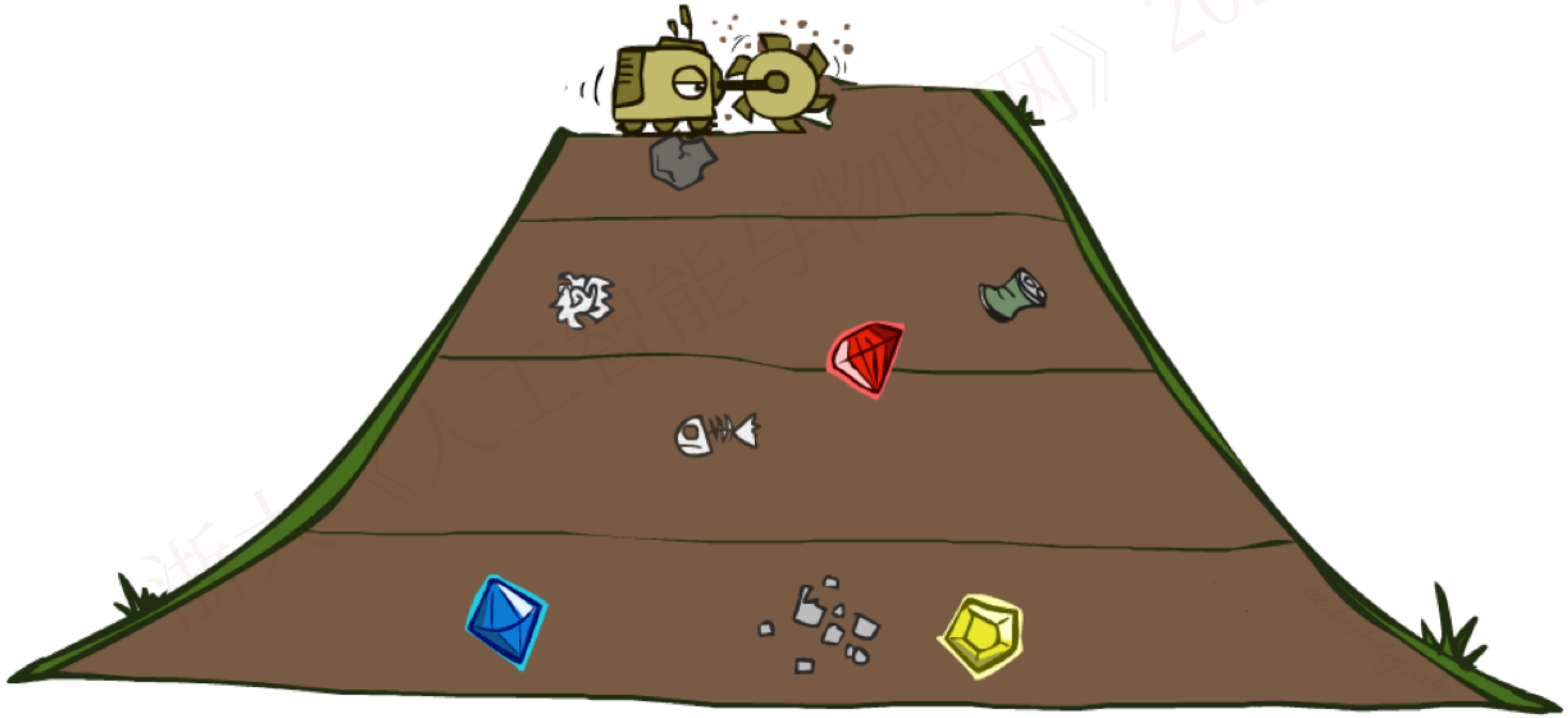
Depth-First Search (DFS) Properties

- What nodes DFS expand?
  - Some left prefix of the tree.
  - Could process the whole tree!
  - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
  - Only has siblings (兄弟节点) on path to root, so O(bm)
- Is it complete?
  - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
  - No, it finds the “leftmost” solution, regardless of depth or cost



浙大《人工智能与物联网》2026夏

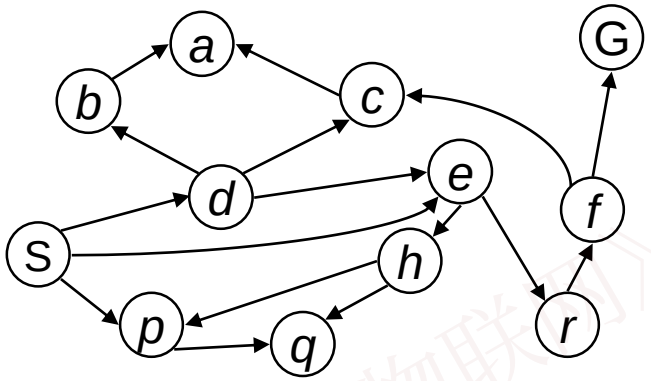
Breadth-First Search

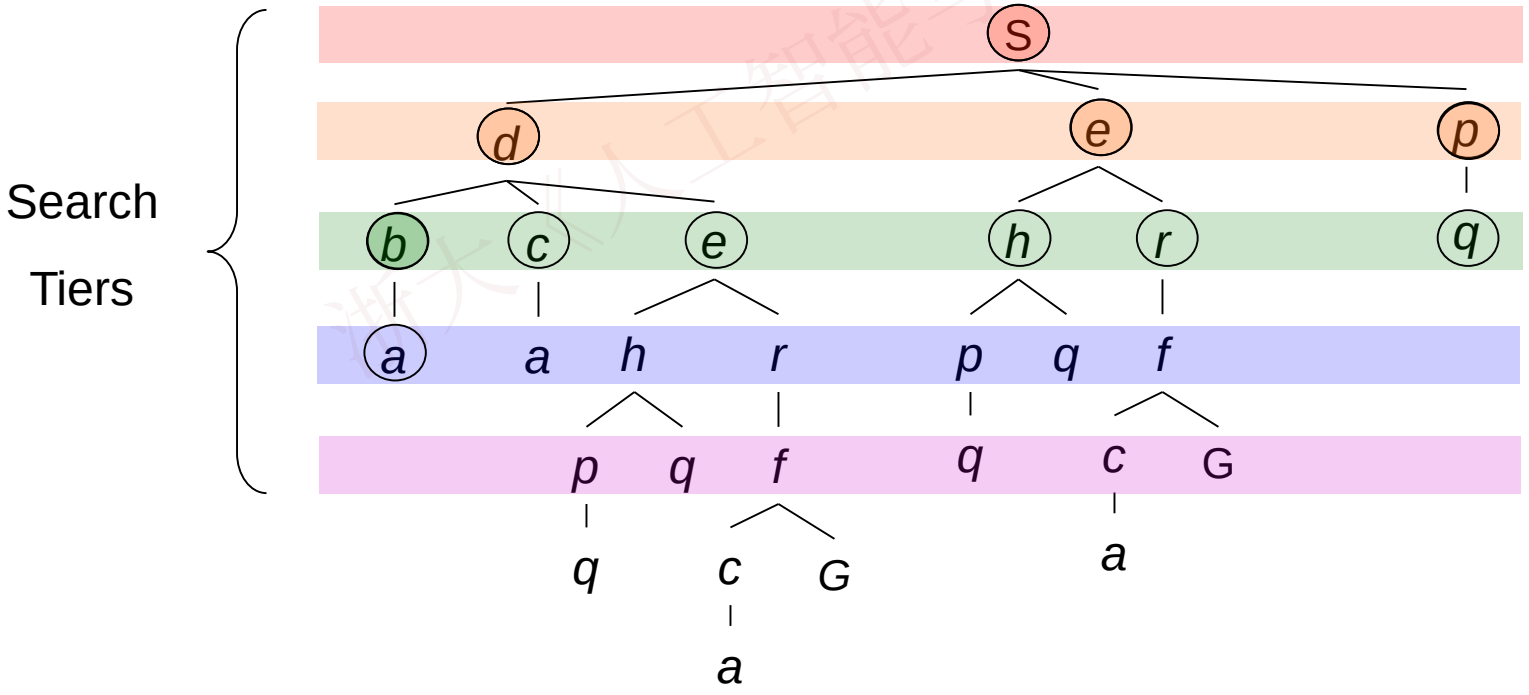


Breadth-First Search

Strategy: expand a shallowest node first

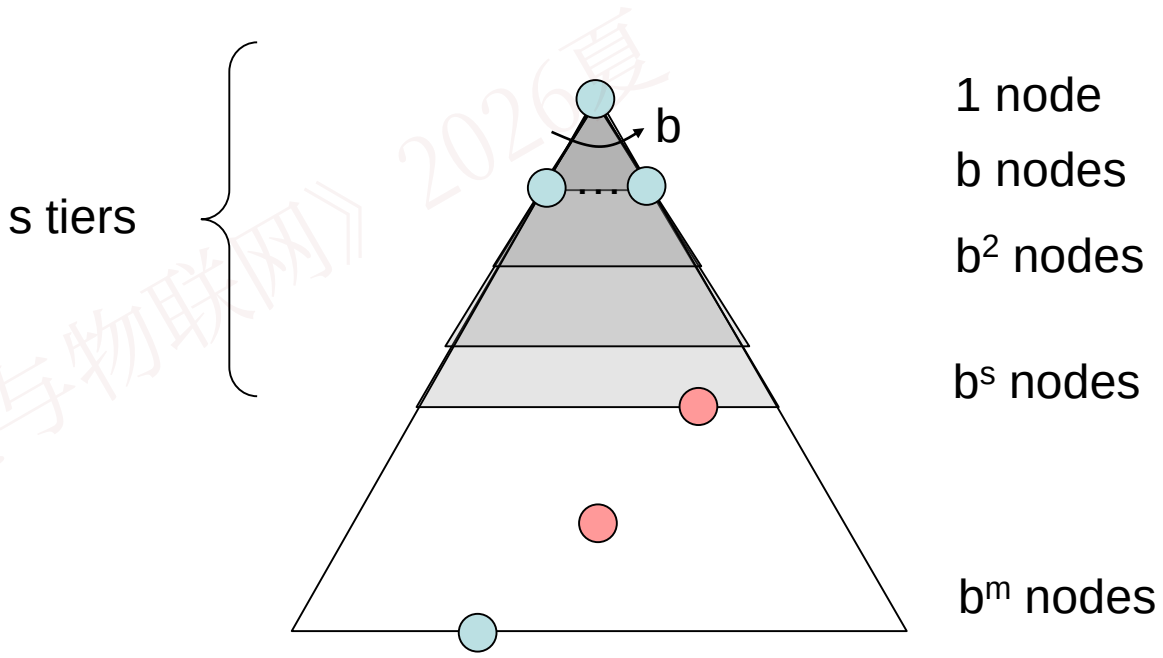
Implementation: Fringe is a FIFO queue





Breadth-First Search (BFS) Properties

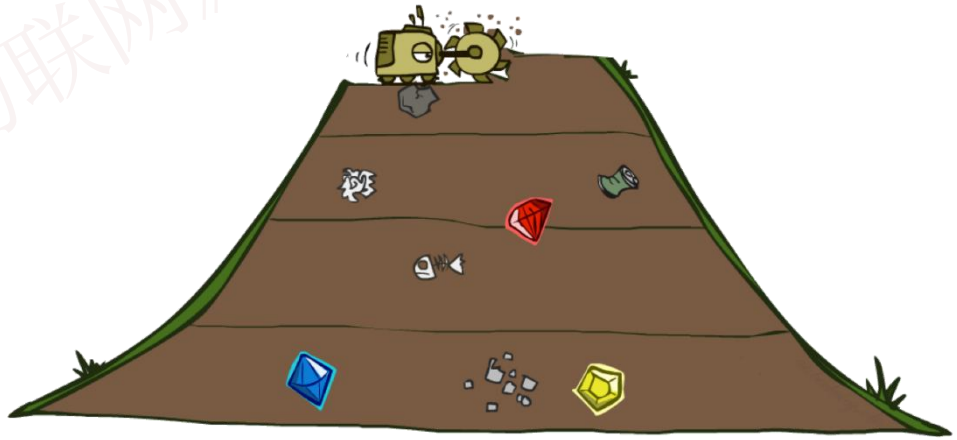
- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be s
  - Search takes time $O(b^s)$
- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^s)$
- Is it complete?
  - s must be finite if a solution exists, so yes!
- Is it optimal?
  - Only if costs are all 1 (more on costs later)



浙大《人工智能与物联网》2026夏

Quiz: DFS vs BFS





浙大人工智能与物联网》2026夏

Quiz: DFS vs BFS

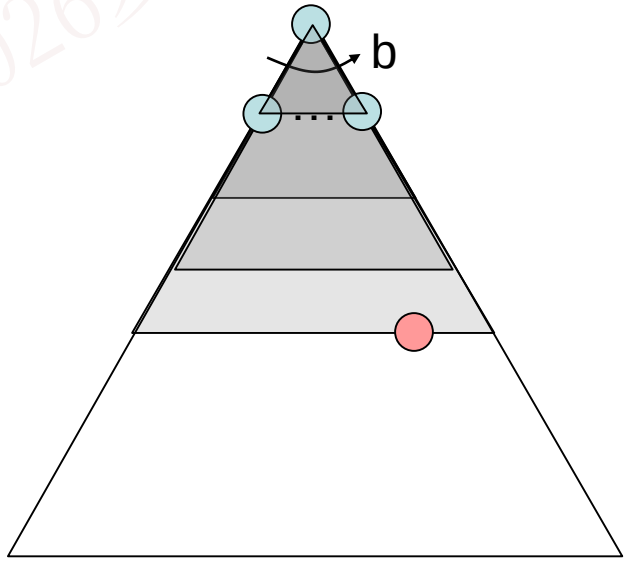
- When will BFS outperform DFS?

- When will DFS outperform BFS?

浙大《人工智能与物联网》2026夏

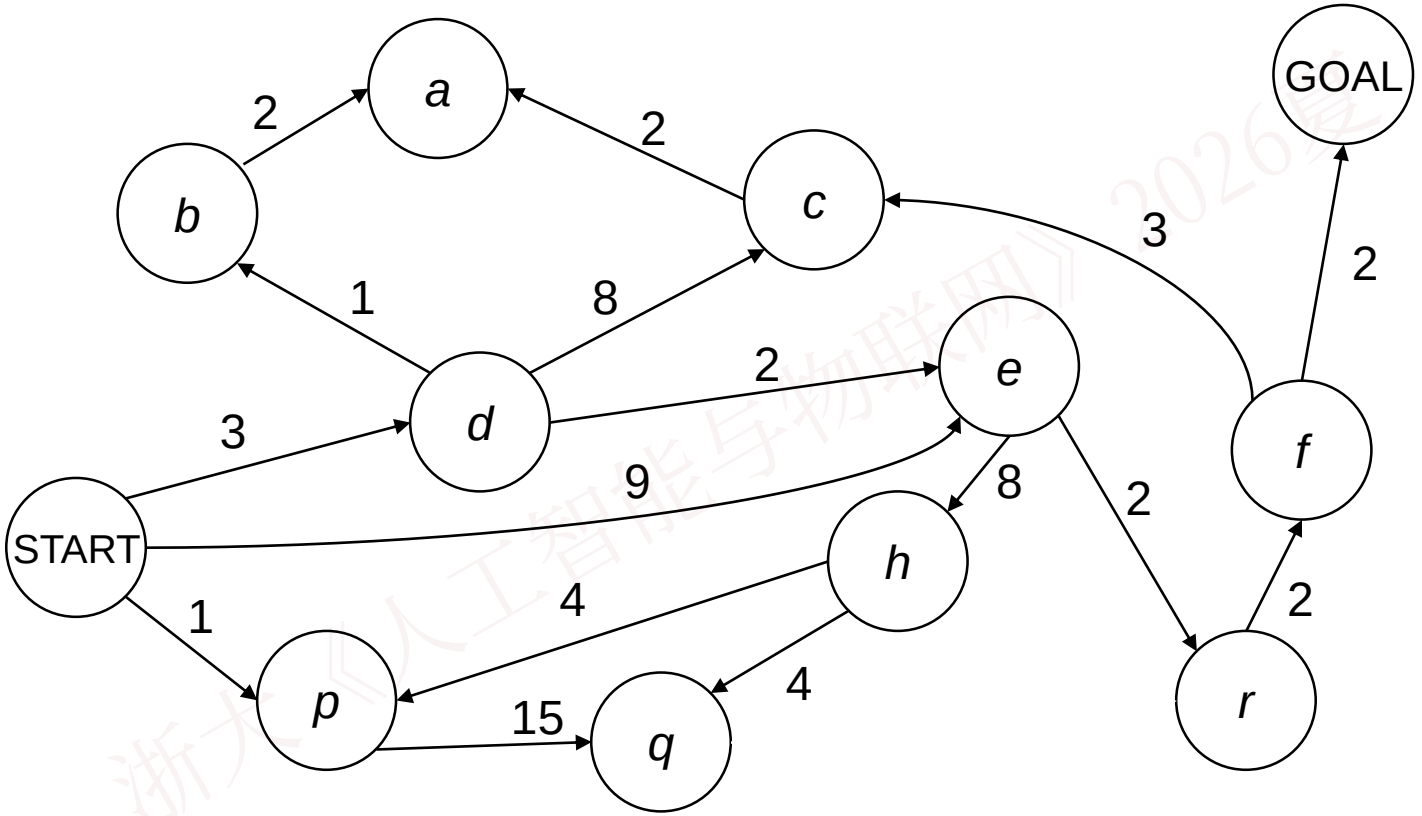
Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. …..
- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!



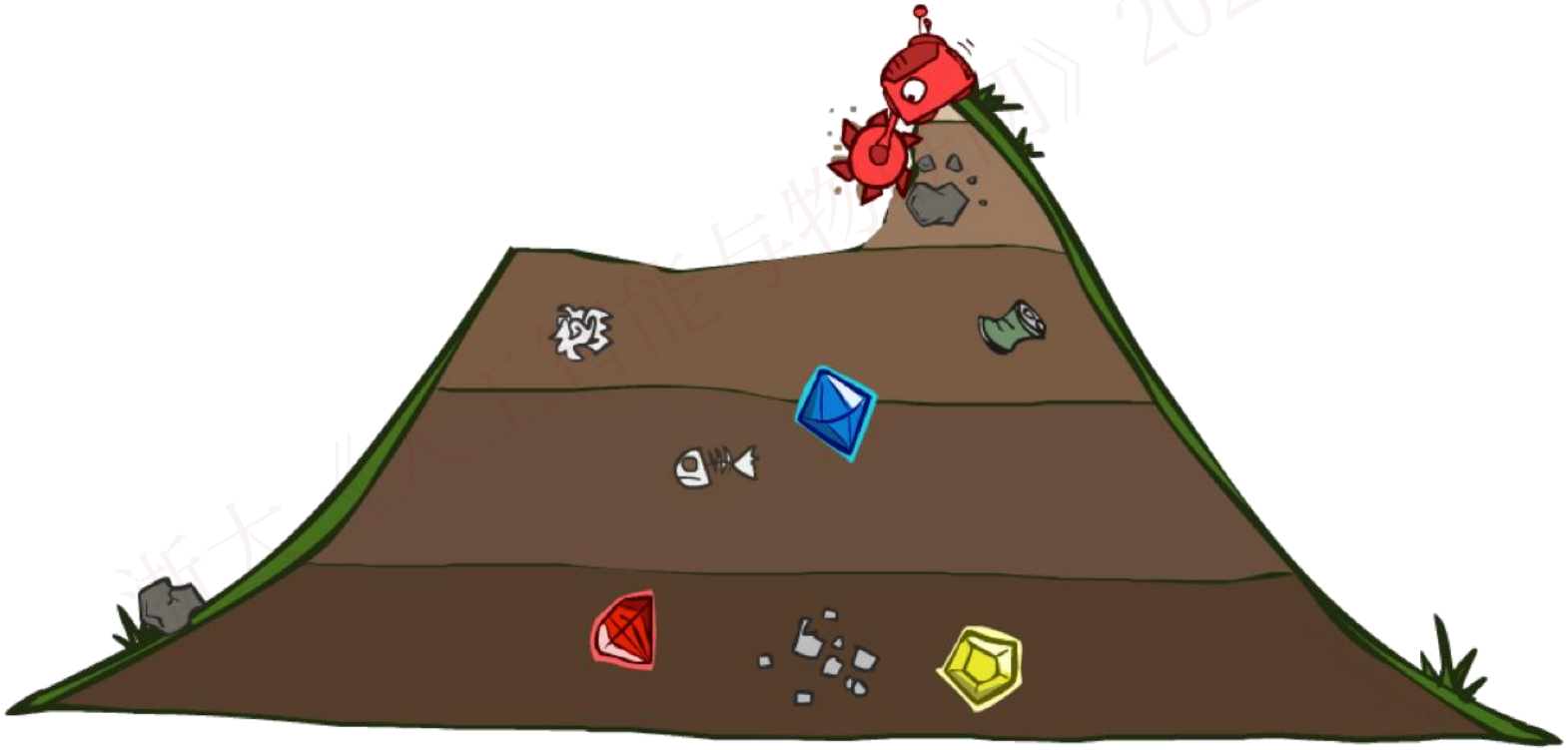
浙大《人工智能与物联网》2026夏

Cost-Sensitive Search



BFS finds the shortest path in terms of number of actions.
It does not find the least-cost path. We will now cover
a similar algorithm which does find the least-cost path.

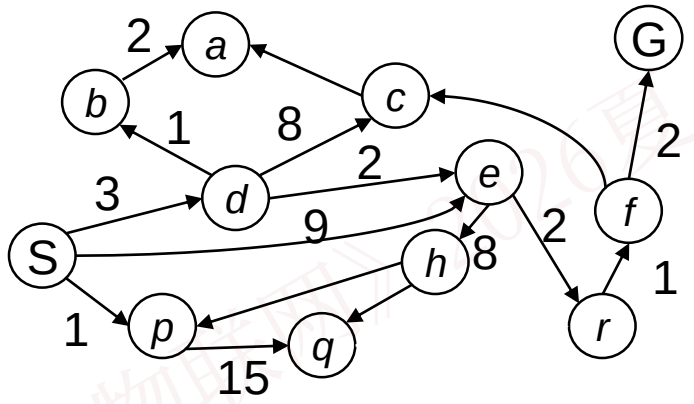
Uniform Cost Search

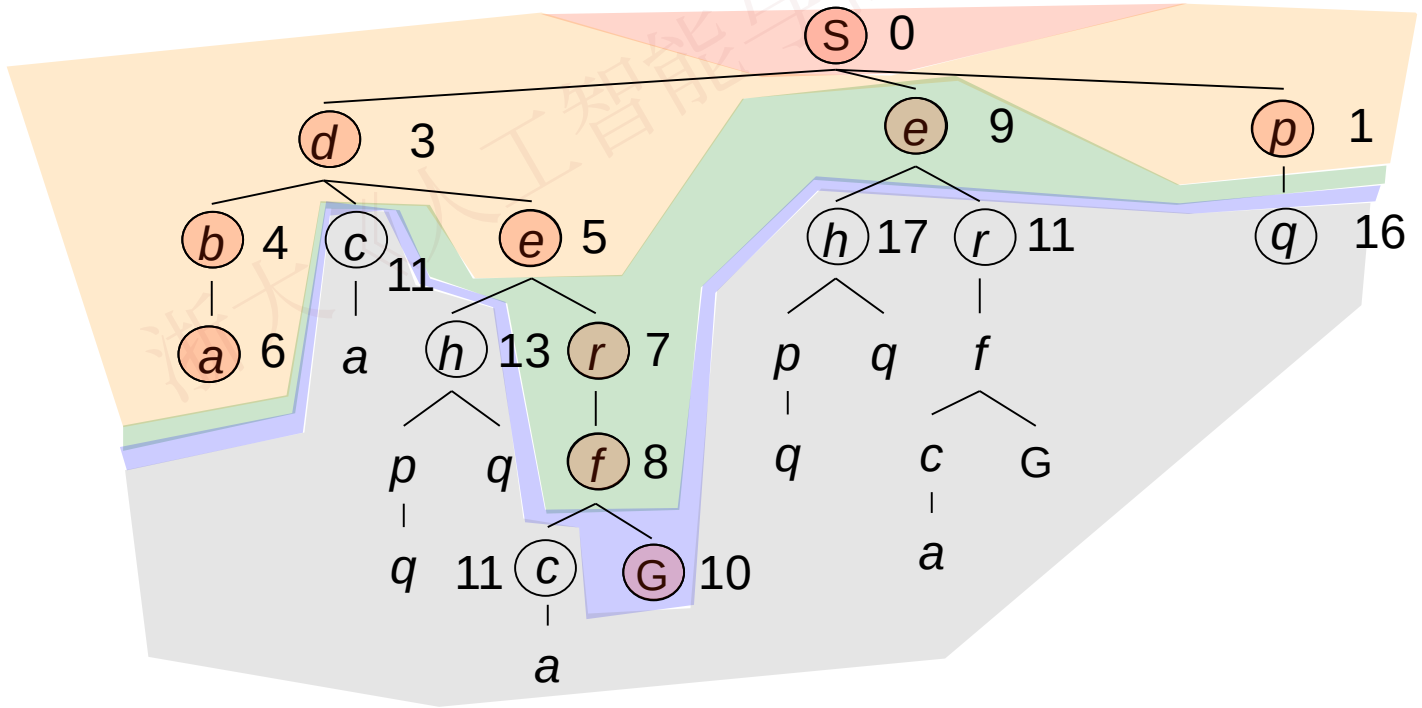


Uniform Cost Search

Strategy: expand a cheapest node (累积代价最小) first:

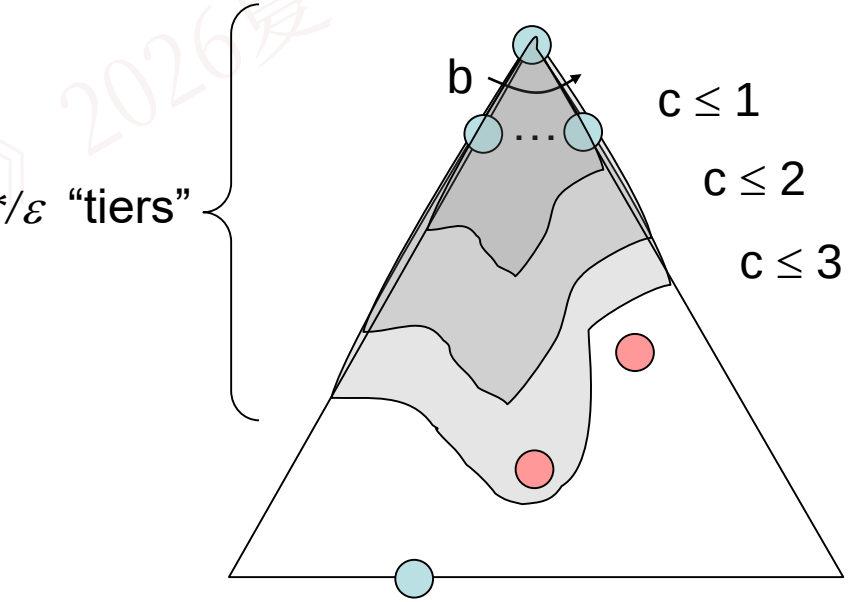
Fringe is a priority queue (priority: cumulative cost)





Uniform Cost Search (UCS) Properties

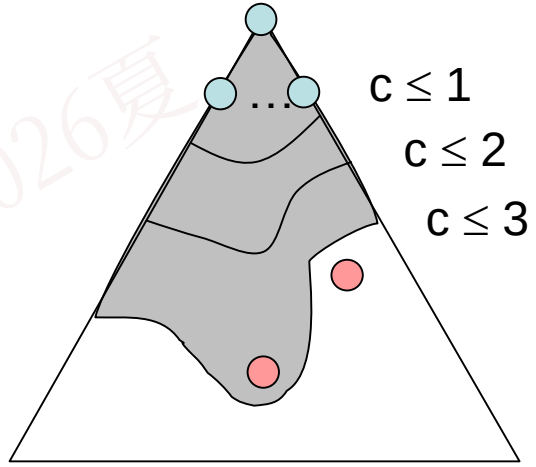
- What nodes does UCS expand?
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs C^* and every edge costs at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time $O(b^{C^*/\varepsilon})$ (exponential in effective depth)
- How much space does the fringe take?
  - Has roughly the last tier, so $O(b^{C^*/\varepsilon})$
- Is it complete?
  - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
  - Yes! (Proof later via A*)



浙大《人工智能与物联网》2026夏

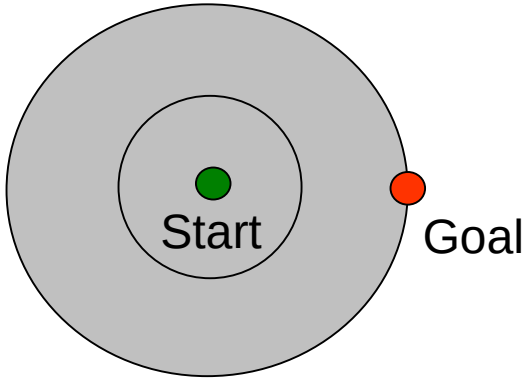
Uniform Cost Issues

- Remember: UCS explores increasing cost contours



- The good: UCS is complete and optimal!

- The bad:
  - Explores options in every “direction”
  - No information about goal location



- We’ll fix that soon!

进入《人工智能与物联网》2026夏

Search and Models

- Search operates over models of the world
  - The agent doesn't actually try all the plans out in the real world!
  - Planning is all "in simulation"
  - Your search is only as good as your models…

